



SEPTQ: A Simple and Effective Post-Training Quantization Paradigm for Large Language Models

Han Liu
Dalian University of Technology
Dalian, China
liu.han.dut@gmail.com

Haotian Gao
Dalian University of Technology
Dalian, China
haotian.dlut@gmail.com

Xiaotong Zhang*
Dalian University of Technology
Dalian, China
zxt.dut@hotmail.com

Changya Li
Dalian University of Technology
Dalian, China
lichangya.dut@gmail.com

Feng Zhang
Peking University
Beijing, China
zfeng.maria@gmail.com

Wei Wang
Shenzhen MSU-BIT University
Shenzhen, China
ehomewang@ieee.org

Fenglong Ma
The Pennsylvania State University
University Park, United States
fenglong@psu.edu

Hong Yu
Dalian University of Technology
Dalian, China
hongyu@dlut.edu.cn

Abstract

Large language models (LLMs) have shown remarkable performance in various domains, but they are constrained by massive computational and storage costs. Quantization, an effective technique for compressing models to fit resource-limited devices while preserving generative quality, encompasses two primary methods: quantization aware training (QAT) and post-training quantization (PTQ). QAT involves additional retraining or fine-tuning, thus inevitably resulting in high training cost and making it unsuitable for LLMs. Consequently, PTQ has become the research hotspot in recent quantization methods. However, existing PTQ methods usually rely on various complex computation procedures and suffer from considerable performance degradation under low-bit quantization settings. To alleviate the above issues, we propose a simple and effective post-training quantization paradigm for LLMs, named SEPTQ. Specifically, SEPTQ first calculates the importance score for each element in the weight matrix and determines the quantization locations in a static global manner. Then it utilizes the mask matrix which represents the important locations to quantize and update the associated weights column-by-column until the appropriate quantized weight matrix is obtained. Compared with previous methods, SEPTQ simplifies the post-training quantization procedure into only two steps, and considers the effectiveness and efficiency simultaneously. Experimental results on various datasets across a suite of models ranging from millions to billions in different quantization

bit-levels demonstrate that SEPTQ significantly outperforms other strong baselines, especially in low-bit quantization scenarios.

CCS Concepts

• **Computing methodologies** → **Natural language processing.**

Keywords

Large Language Models, Post-Training Quantization

ACM Reference Format:

Han Liu, Haotian Gao, Xiaotong Zhang, Changya Li, Feng Zhang, Wei Wang, Fenglong Ma, and Hong Yu. 2025. SEPTQ: A Simple and Effective Post-Training Quantization Paradigm for Large Language Models. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '25)*, August 3–7, 2025, Toronto, ON, Canada. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3690624.3709287>

1 Introduction

Large language models (LLMs) [7, 20, 26, 32], based on the transformer architecture [38], have demonstrated exceptional performance across various downstream tasks. Despite their impressive capabilities, the immense number of parameters in these models presents significant deployment challenges on end-user devices due to high computational and storage demands. For instance, the GPT-175B model [4], which contains 175 billion parameters, requires approximately 350GB of storage space and more than 5 A100 GPUs (each with 80GB of memory) for inference. This underscores the urgent necessity for effective compression strategies for large language models.

Quantization has become a widely embraced technique to alleviate the storage and computational overhead of deep learning models [1, 22, 25, 40], and its core idea is to convert model parameters from floating-point to lower-bit integer representation. In general, existing quantization methods can be divided into two categories: quantization aware training (QAT) and post-training quantization (PTQ). For QAT, it integrates the quantization objective into the model training process seamlessly, thus requiring

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

KDD '25, August 3–7, 2025, Toronto, ON, Canada

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1245-6/25/08

<https://doi.org/10.1145/3690624.3709287>

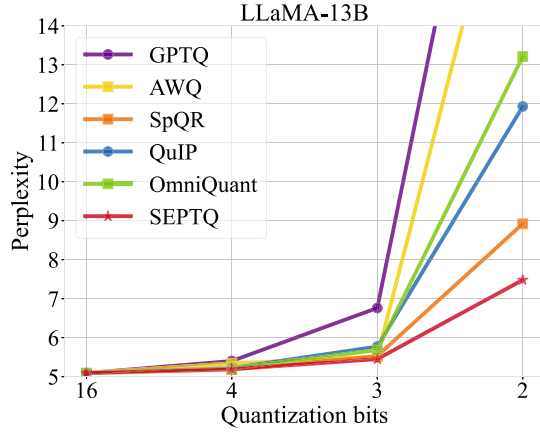


Figure 1: The perplexity results of widely-used quantization methods on LLaMA-13B, reported on WikiText2 (the lower value is better). The perplexity values of GPTQ and AWQ at 2-bit level are too large to display. All the approaches suffer from significant performance degradation when quantizing models to 2-bit level, and our proposed SEPTQ outperforms other baselines obviously.

additional retraining or fine-tuning. Its substantial training expense renders it impractical for LLMs. For PTQ, it diminishes the storage and computational complexity of the models without necessitating modifications to the LLM architecture or requiring a retraining process. Its simplicity and efficiency make it a common choice in existing quantization methods for LLMs.

Recently, several methods have been proposed for post-training quantization [9, 28, 43]. In particular, GPTQ [14] is a novel layer-wise quantization method based on approximate second-order information which aims to solve the problem of minimizing the layer-wise squared error. It can compress models to 3 bits or 4 bits without significant loss of accuracy. AWQ [24] is an activation-aware method by considering the significance of weight channels corresponding to larger activation magnitudes. SpQR [21] identifies sensitive weights and store them in higher precision, while compressing all other weights to 3-4 bits. QuIP [5] introduces an adaptive rounding procedure which minimizes a quadratic proxy of the weight error and can guarantee the incoherence of the weight and Hessian matrices. It is the first method which can obtain stable results in the 2-bit level compression. These methods have demonstrated considerable efficacy in reducing both computational and memory overhead for LLMs. Nevertheless, they either rely on intricate optimization procedures or suffer from substantial performance degradation in scenarios involving low-bit quantization [16, 39, 41], and Figure 1 shows an example that the efficacy of different PTQ methods rapidly diminishes when the quantization bit-width is extremely low.

In this paper, we propose a simple and effective paradigm for post-training quantization, named SEPTQ, which not only ensures the effectiveness of the quantization process but also maintains its efficiency. SEPTQ is comprised of two modules: determining the quantization location and quantizing the model weights. For

the first component, we adopt a static global strategy to identify the important parameters for each weight matrix and then obtain the corresponding mask matrix which represents the quantization locations. For the second component, we utilize the mask matrix that serves as a guide to quantize and update the associate weights column-by-column, where the update formula is derived under the constraint of the mask matrix and is able to precisely compensate for the rounding error that are inevitable when weights are quantized. Extensive experiments on a broad range of datasets and a diverse set of models, spanning from modest millions to immense billions of parameters, have unequivocally revealed the superiority of SEPTQ over other formidable baselines. Notably, in the low-bit (2-bit) quantization scenarios, where the challenge of maintaining model accuracy is most acute, SEPTQ has demonstrated significant performance gains, thereby further validating its effectiveness and robustness in the domain of model quantization.

2 Related Work

Quantization aims to reduce the storage and computational overhead of deep learning models, which has become a widely accepted technique in model compression. Unlike traditional floating-point representation, quantization converts these numbers into integers or other discrete forms, thus significantly decreasing the storage requirement and computational complexity [46]. Existing model quantization methods can be divided into two main categories based on when quantization is applied: **Quantization Aware Training (QAT)** and **Post-Training Quantization (PTQ)**.

2.1 Quantization Aware Training

QAT achieves the quantization during the model training [8, 18, 36]. LLM-QAT [27] is a quantization-aware training approach that employs data-free distillation using data generated by a pre-trained model, bypassing data collection issues. OmniQuant [35] achieves low-bit quantization by introducing learnable weight clipping (LWC) to optimize the clipping threshold and learnable equivalent transformation (LET) to shift the quantization challenge from activations to weights, effectively handling activation outliers. Although the QAT methods often have good performance, their processes are often tedious and time-consuming.

2.2 Post-Training Quantization

PTQ quantizes the model after it has completed training [11, 13, 23], which aims to reduce the storage and computational complexity of LLMs without modifying the model architecture or requiring a retraining process. Compared with QAT, PTQ is simple and efficient in achieving model compression. Optimal brain quantization (OBQ) [12] extends the traditional second-order weight pruning framework of optimal brain surgeon (OBS) [15, 19]. It quantizes weights based on quantization errors and achieves good results on smaller models without retraining. GPTQ [14] introduces a layer-wise quantization technology based on approximate second-order information. It quantizes the weights column-wise and updates the remaining weights according to the OBQ method until all weights are quantized. AWQ [24] looks for optimal per-channel scaling factors to protect these outlier (important) weights by looking at activation patterns rather than the weights themselves. However, if

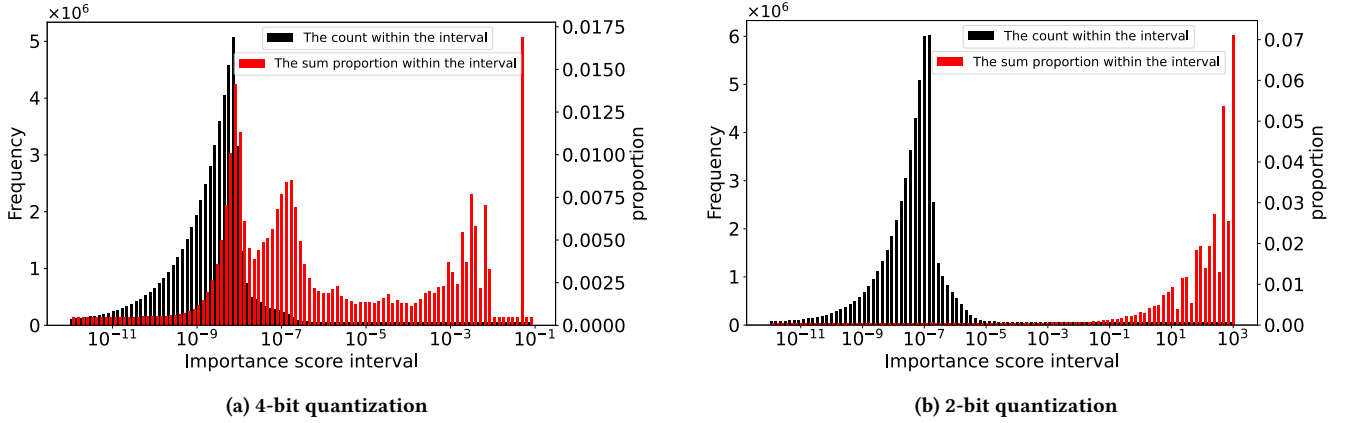


Figure 2: The frequency and proportion distributions of the importance scores in different quantization settings. In 4-bit quantization (a), a small number of weights have high importance scores and account for almost half of the importance score of the entire weight matrix. In 2-bit quantization (b), a small number of weights have extremely high importance scores and account for almost all of the importance score of the entire weight matrix.

the scaling factor is too large, it will increase the quantization loss of non-outlier weights, while if the scaling factor is too small, it cannot protect these outlier weights well. SpQR [21] achieves near-lossless compression at different model sizes by identifying and isolating those outlier weights that cause particularly large quantization errors and storing them with higher precision. However, it uses the dynamic local strategy to find the outlier weights which is time-consuming and suboptimal. QuIP [5] employs an adaptive rounding procedure to minimize a quadratic approximation target and then performs efficient pre- post-processing to ensure the incoherence of the weight and Hessian matrices by multiplying them by a Kronecker product of random orthogonal matrices. It is a powerful method and achieves 2-bit level quantization, but its computation cost is a little expensive.

3 Problem Formulation

In this paper, we focus on the problem of post-training quantization [6, 17, 31], and aims to design a simple and effective post-training quantization framework for large language models. Specifically, let $\mathbf{W} \in \mathbb{R}^{d_{row} \times d_{col}}$ denote the weight matrix of a linear layer and $\mathbf{X} \in \mathbb{R}^{d_{col} \times n}$ represent an input matrix, where n is the number of samples. The whole goal is to discover a compressed version of $\mathbf{W}$ (denoted as $\hat{\mathbf{W}}$), which can minimize the squared error associated with the full precision layer output. Formally,

$$\operatorname{argmin}_{\hat{\mathbf{W}}} \|\mathbf{W}\mathbf{X} - \hat{\mathbf{W}}\mathbf{X}\|_F^2, \quad \text{s.t. } \hat{\mathbf{W}} = \operatorname{quant}(\mathbf{W}), \quad (1)$$

where $\|\cdot\|_F$ is the Frobenius norm, and $\operatorname{quant}(\cdot)$ refers to the round-to-nearest (RTN) method [42], which can round a model weight to the nearest value on the quantization grid. In particular, if $\mathbf{W}_{i,j}$ denotes the i -th row and j -th element of $\mathbf{W}$, then $\hat{\mathbf{W}}_{i,j}$ can be calculated as:

$$\hat{\mathbf{W}}_{i,j} = S(\mathbf{W}_{i,j}^q - Z), \quad \mathbf{W}_{i,j}^q = \operatorname{clip}(\lfloor \frac{\mathbf{W}_{i,j}}{S} \rceil + Z, 0, 2^N - 1), \quad (2)$$

where S is the quantization scale parameter, Z is the zero point parameter, N is the number of quantization bits, $\lfloor \frac{\mathbf{W}_{i,j}}{S} \rceil$ is the rounding

function, and $\operatorname{clip}(\cdot)$ is the truncation function which can guarantee that the value is the range of 0 to $2^N - 1$.

4 The Proposed Paradigm

The proposed quantization paradigm consists of two components: determining the quantization location and quantizing the model weights. In this section, we will introduce these modules in details.

4.1 Determining the Quantization Location

In general, different weight parameters in a neural network contribute unequally [21]. Therefore, for the quantization task, it is a crucial step to determine the quantization location (weight parameter). Intuitively, if quantizing a weight parameter leads to a large rounding error, the weight parameter can be treated as an important parameter which tends to be reserved. In contrast, if quantizing a weight parameter affects the rounding error slightly, the weight parameter can be treated as an unimportant parameter which should be quantized.

Furthermore, if we utilize the square error between $\mathbf{W}\mathbf{X}$ and $\hat{\mathbf{W}}\mathbf{X}$ to obtain the rounding error, we can have the following formula to calculate the importance score $s_{i,j}$ for each element $\mathbf{W}_{i,j}$ in $\mathbf{W}$:

$$s_{i,j} = \min_{\hat{\mathbf{W}}(i,j)} \|\mathbf{W}\mathbf{X} - \hat{\mathbf{W}}(i,j)\mathbf{X}\|_F^2, \quad (3)$$

$$\text{s.t. } \hat{\mathbf{W}}(i,j) = \operatorname{quant}(\mathbf{W}(i,j)),$$

where $\hat{\mathbf{W}}(i,j)$ denotes the matrix $\mathbf{W}$ with only $\mathbf{W}_{i,j}$ is quantized and unquantized elements in the i -th row are updated. By leveraging Taylor formula and the Lagrangian multiplier method (The detailed proof procedure can refer to Appendix A), the importance score $s_{i,j}$ can be derived as:

$$s_{i,j} = \frac{(\mathbf{W}_{i,j} - \operatorname{quant}(\mathbf{W}_{i,j}))^2}{2[\mathbf{X}\mathbf{X}^T]_{j,j}^{-1}}. \quad (4)$$

By utilizing Eq. (4), in terms of a particular input matrix $\mathbf{X}$, we can obtain the importance score of each element in $\mathbf{W}$. Following

that, we sort all the importance scores in the descending order, and select the top $p\%$ elements as reserved weight parameters, and the remaining $1 - p\%$ elements as quantized weight parameters. In this paper, we usually set the ratio p to be less than or equal to 1%.

In addition, we further analyze the distribution of important scores. Specifically, we take the output projection matrix of the fourth layer in the OPT-30B model as the example to explore the distribution of important scores (For more experimental results, please refer to the Appendix C). We first calculate the importance score for each weight of the output projection matrix, and then draw the frequency distribution histogram and proportion histogram of these scores, which are shown in Figure 2. The black bars represent the number of weights within the importance score interval, and the red bars represent the proportion of the sum of the importance scores within the importance score interval to the sum of all importance scores.

A salient feature observed from Figure 2 (a) is the prevalence of lower importance scores, with a minority exhibiting higher values. Despite their scarcity, these high-scoring weights exert a disproportionately large influence on the total importance score. The analysis extends to Figure 2 (b), revealing that the vast majority of importance scores fall below 10^{-7} . Interestingly, despite their numerical abundance, these scores contribute minimally to the overall sum, highlighting a more pronounced long-tail effect in 2-bit quantization compared to its 4-bit counterpart. This exacerbation of the long-tail phenomenon underscores the critical role of a select few high-importance weights in the quantization process, particularly as the quantization bit-width diminishes.

The implications of these findings are twofold. Firstly, the presence of a few high-importance weights indicates that their impact on the quantization process is non-trivial. Secondly, as quantization bit-width decreases, the influence of these critical weights becomes more pronounced. Effective weight selection is crucial for maintaining model performance in quantization. Based on these insights, by preserving the quantization of important weights and selectively quantizing the less critical ones, the precision of compression can be enhanced. Moreover, the benefits of this strategy increase as the quantization bit width decreases.

For the ease of representing the quantization locations, we introduce the mask matrix $\mathbf{M}$ to express whether an element in the weight matrix $\mathbf{W}$ is selected as the quantization location. If $\mathbf{M}_{i,j} = 0$, it indicates $\mathbf{W}_{i,j}$ needs to be quantized. And if $\mathbf{M}_{i,j} = 1$, it indicates $\mathbf{W}_{i,j}$ needs to be reserved.

4.1.1 Static Strategy vs. Dynamic Strategy. By static strategy, we mean that the importance scores of each weight matrix are calculated at once. By dynamic strategy, we mean that the importance scores of each weight matrix are calculated sequentially (column-by-column), as previous works like GPTQ [14] update some corresponding weight parameters to compensate for rounding errors after quantizing each column. To analyze which strategy is more appropriate, we attempt to visualize the determined quantization locations (the mask matrix $\mathbf{M}$) by static strategy and dynamic strategy. Specifically, we take the 7-th layer weight matrix of the OPT-30B model [45] as example and set the $p = 1\%$. As the layer dimension is too large, we divide the whole matrix into different blocks with the size of 128 rows $\times$ 128 columns, and sum the element values of each

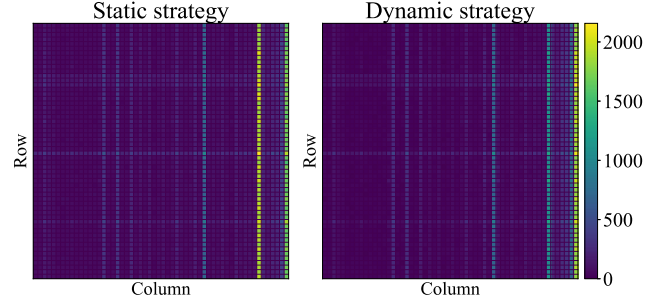


Figure 3: The visualization of determined quantization location distribution by static strategy (ours) and dynamic strategy (GPTQ). The value of each block is the sum of the values in the corresponding locations of the mask matrix $\mathbf{M}$.

block in $\mathbf{M}$. Figure 3 shows the the determined quantization location distribution. It can be seen that the overall distribution situation is very similar, which indicates that no matter using static strategy or dynamic strategy to determine the quantization locations, the performance will be stable. The dynamic strategy will inevitably add extra computation cost which is caused by updating weight parameters. Therefore, we employ the static strategy to determine the quantization locations.

4.1.2 Global Strategy vs. Local Strategy. By global strategy, we mean that selecting the top $p\%$ elements from the whole matrix as reserved weight parameters. By local strategy, we mean that selecting the top $p\%$ elements from each block as reserved weight parameters. As illustrated in Figure 3, we can get that the important weight parameters tend to cluster in some specific rows or columns. That is to say, some blocks clearly contain a large amount of important weight parameters, while some blocks do not have any important weight parameters which are worth preserving. Based on the above observation, we utilize the global strategy to determine the quantization locations.

4.2 Quantizing the Model Weights

This procedure consists of two steps: quantizing the model weights in selected quantization locations and updating the model weights in the non-quantization locations.

In terms of quantizing the model weights in selected quantization locations, as the computation complexity of directly solving Eq. (1) is excessively high, inspired by [14], we quantize the model weights in a column-by-column manner and the specific quantization computation formula can refer to Eq. (2).

In terms of updating the model weights in the non-quantization locations, we calculate the increment for the non-quantization weights to compensate the rounding error caused by quantized weights. In particular, after obtaining the mask matrix which can represent the quantization locations, we can modify the constraint in Eq. (1) as:

$$\hat{\mathbf{W}} = \text{quant}(\mathbf{W}) + \mathbf{M} \odot (\mathbf{W} - \text{quant}(\mathbf{W})). \quad (5)$$

According to Eq. (5), it is easy to find that if $\mathbf{M}_{i,j} = 0$, $\hat{\mathbf{W}}_{i,j} = \text{quant}(\mathbf{W}_{i,j})$. And if $\mathbf{M}_{i,j} = 1$, $\hat{\mathbf{W}}_{i,j} = \mathbf{W}_{i,j}$. Furthermore, we employ

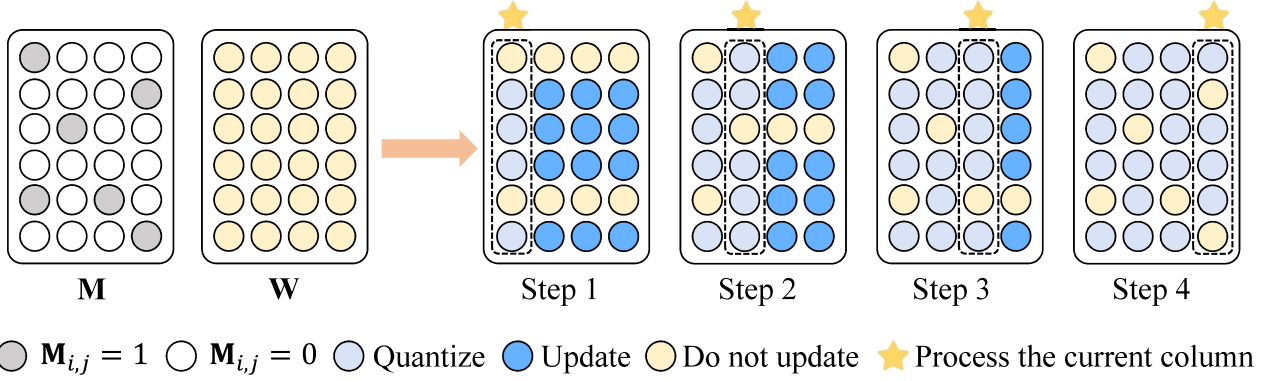


Figure 4: The visualization of quantizing the model weights. Given the original weight matrix $\mathbf{W}$ and the mask matrix $\mathbf{M}$, we process the matrix $\mathbf{W}$ in a column-by-column manner. If $\mathbf{M}_{i,j} = 1$, the corresponding weights and the subsequent weights in the same row both remain unchanged (light yellow). If $\mathbf{M}_{i,j} = 0$, the corresponding weights are quantized (light blue) and the subsequent weights in the same row are updated to compensate for the quantization loss (dark blue).

the similar method in [12] to solve the Eq. (1) with the constraint in Eq. (5) (The detailed proof procedure can refer to Appendix B).

Specifically, assume that when processing the j -th column in $\mathbf{W}$, the i -th element of this column is quantized, then we need to update the corresponding weights in the i -th row and greater than the j -th column of $\mathbf{W}$. we let $\delta = (\hat{\mathbf{W}}_{i,:} - \mathbf{W}_{i,:})^\top$ denote the increment of the weight of the i -th row, according to Eq. (5), we can get the constraint condition of the i -th row as

$$\mathbf{e}_j^T \delta + (\mathbf{W}_{i,j} - \hat{\mathbf{W}}_{i,j}) = 0, \quad (6)$$

where $\mathbf{e}_j$ represents the unit vector with 1 in the j -th position. This problem has the following Lagrangian[12]:

$$L(\delta, \lambda) = \frac{1}{2}(\delta^T \mathbf{H} \delta) + \lambda(\mathbf{e}_j^T \delta + (\mathbf{W}_{i,j} - \hat{\mathbf{W}}_{i,j})). \quad (7)$$

By solving the derivatives of δ and λ , we can get:

$$\delta = -\frac{\mathbf{W}_{i,j} - \hat{\mathbf{W}}_{i,j}}{[\mathbf{H}^{-1}]_{jj}} [\mathbf{H}^{-1}]_{:,j}, \quad (8)$$

where $\mathbf{H} = 2\mathbf{X}\mathbf{X}^T$, and we only need take greater than the j -th elements of δ to update the associated weights. To illustrate the above procedure clearly, we take an example to visualize the compression pipeline of our algorithm in Figure 4.

4.3 The Overall Algorithm

The detailed algorithm procedure is shown in Algorithm 1. We implement our algorithm based on the GPTQ framework [14]. Compression is performed per linear layer. Specifically, given a weight matrix $\mathbf{W}$ and a calibration dataset $\mathbf{X}$, SEPTQ first computes the Hessian matrix and the inverse of the Hessian matrix approximated through Cholesky decomposition. Then it calculates the mask $\mathbf{M}$ in a static and global manner. Subsequently, some associated weights are quantized and updated. During the update process, we apply our algorithm to each block based on the lazy batch-update, and perform the global update after a block is processed [14]. Finally, it can obtain the quantized weight matrix $\hat{\mathbf{W}}$.

Algorithm 1 The SEPTQ Algorithm

Input: The weight matrix $\mathbf{W}$, the calibration dataset $\mathbf{X}$, the ratio parameter p , the blocksize B

Output: The quantized weight matrix $\hat{\mathbf{W}}$

```

1:  $\mathbf{Q} \leftarrow \mathbf{0}_{d_{\text{row}} \times d_{\text{col}}}$  // Save the quantized weights
2:  $\mathbf{E} \leftarrow \mathbf{0}_{d_{\text{row}} \times B}$  // Initialize the block quantization errors
3:  $\mathbf{M} \leftarrow \mathbf{1}_{d_{\text{row}} \times d_{\text{col}}}$  // Initialize the static global mask matrix
4:  $\mathbf{H}^{-1} \leftarrow (2\mathbf{X}\mathbf{X}^T + \lambda\mathbf{I})^{-1}$ 
5:  $\mathbf{H}^{-1} \leftarrow \text{Cholesky}(\mathbf{H}^{-1})$  //Use Cholesky decomposition
6:  $\mathbf{W}^Q \leftarrow \text{quant}(\mathbf{W})$ 
7:  $\mathbf{M} \leftarrow \text{mask of } p\% \text{ weights by Eq. (4)}$ 
8: for  $i = 0, B, 2B, \dots$  do
9:   for  $j = i, \dots, i + B - 1$  do
10:     $\mathbf{Q}_{:,j} \leftarrow \text{quant}(\mathbf{W}_{:,j})$ 
11:     $\mathbf{Q}_{:,j} \leftarrow \mathbf{Q}_{:,j} + \mathbf{M}_{:,j} \odot (\mathbf{W}_{:,j} - \mathbf{Q}_{:,j})$  // Calculate by Eq. (5)
12:     $\mathbf{E}_{:,j-i} \leftarrow (\mathbf{W}_{:,j} - \mathbf{Q}_{:,j}) / [\mathbf{H}^{-1}]_{jj}$  // Quantization error
13:     $\mathbf{W}_{:,j:(i+B)} \leftarrow \mathbf{W}_{:,j:(i+B)} - \mathbf{E}_{:,j-i} \mathbf{H}_{j,j:(i+B)}^{-1}$  // Update the weights by Eq. (8)
14:   end for
15:    $\mathbf{W}_{:, (i+B):} \leftarrow \mathbf{W}_{:, (i+B):} - \mathbf{E} \mathbf{H}_{i:(i+B), (i+B):}^{-1}$ 
16: end for
17:  $\hat{\mathbf{W}} \leftarrow \mathbf{Q}$ 

```

5 Experiments

5.1 Experimental Settings

To validate the superiority of our proposed method, we follow [14] to conduct two types of experiments: perplexity experiments and accuracy experiments. Following [5, 14], as during the experimental procedure, each model does not see any task-specific data, and the accuracy experiments are also named as zero-shot experiments.

5.1.1 Models and Datasets. For perplexity experiments, we evaluate different models by quantizing the OPT model family [45] from 125 millions to 66 billions on C4 [33], WikiText2 [30] and PTB [29]

Table 1: The perplexity results of the OPT model family on the C4 dataset (The lower value is better). “Full” means the full precision method (the original model). For SEPTQ, it is exactly 2.1-bit, 3.1-bit and 4.1-bit respectively.

Method	Bits	125M	350M	1.3B	2.7B	6.7B	13B	30B	66B
Full	16	26.56	22.59	16.07	14.34	12.71	12.06	11.44	10.99
GPTQ	2	2381.23	6329.39	4342.76	3915.16	522.25	125.88	29.06	253.49
QuIP		177.40	320.00	29.78	38.07	21.62	16.60	13.55	17.64
SEPTQ		53.75	46.33	21.64	18.18	15.19	13.97	12.80	11.96
GPTQ	3	42.01	31.33	21.65	18.20	17.23	13.36	12.22	13.79
QuIP		30.92	25.48	17.12	15.63	13.30	12.39	11.66	11.19
SEPTQ		30.37	25.18	16.99	14.97	13.05	12.32	11.61	11.11
GPTQ	4	29.33	24.62	16.98	15.01	13.18	12.26	11.56	11.25
QuIP		27.63	23.23	16.38	14.55	12.86	12.16	11.50	11.03
SEPTQ		27.78	23.23	16.37	14.55	12.82	12.13	11.49	11.02

Table 2: The perplexity results of the OPT model family on the WikiText2 dataset (The lower value is better). “Full” means the full precision method (the original model). For SEPTQ, it is exactly 2.1-bit, 3.1-bit and 4.1-bit respectively.

Method	Bits	125M	350M	1.3B	2.7B	6.7B	13B	30B	66B
Full	16	27.65	22.00	14.63	12.47	10.86	10.13	9.56	9.34
GPTQ	2	4761.03	17558.09	8293.39	8575.29	3110.27	303.77	54.52	619.32
QuIP		347.40	672.30	41.64	2998.00	22.33	16.02	11.48	13.86
SEPTQ		68.62	54.99	20.11	16.20	12.97	11.82	10.75	9.93
GPTQ	3	52.94	33.85	21.82	16.95	14.94	11.80	10.35	13.69
QuIP		34.22	25.19	16.21	17.44	11.51	10.50	9.79	9.41
SEPTQ		33.77	24.83	15.46	12.74	11.14	10.33	9.61	9.27
GPTQ	4	31.31	24.03	15.50	12.84	11.37	10.31	9.61	9.42
QuIP		33.35	22.50	14.88	12.39	10.98	10.21	9.60	9.42
SEPTQ		29.00	22.78	14.88	12.46	10.98	10.22	9.54	9.27

Table 3: The perplexity results of the OPT model family on the PTB dataset (The lower value is better). “Full” means the full precision method (the original model). For SEPTQ, it is exactly 2.1-bit, 3.1-bit and 4.1-bit respectively.

Method	Bits	125M	350M	1.3B	2.7B	6.7B	13B	30B	66B
Full	16	38.99	31.07	20.29	17.97	15.77	14.52	14.04	13.36
GPTQ	2	4350.98	11670.97	7472.98	6787.27	3195.70	246.22	93.02	630.42
QuIP		430.00	744.20	47.72	63.59	31.73	21.64	17.40	17.50
SEPTQ		96.09	74.70	31.23	24.69	19.28	17.88	16.40	14.93
GPTQ	3	73.58	45.95	30.85	24.76	22.31	16.74	15.38	26.46
QuIP		47.34	35.65	22.76	20.79	16.52	15.05	14.37	13.55
SEPTQ		48.21	36.11	22.24	18.89	16.27	14.88	14.25	13.51
GPTQ	4	45.60	34.15	22.04	19.15	16.49	14.85	14.26	13.81
QuIP		40.80	32.57	20.87	18.42	15.93	14.69	14.18	13.40
SEPTQ		41.26	32.65	20.78	18.25	15.87	14.67	14.12	13.40

datasets. For zero-shot experiments, we measure different models by quantizing the popular LLaMA model [37] from 7 billions to 30 billions on PIQA [3], ARC-easy and ARC-challenge [2], HellaSwag

[44] and WinoGrande [34] datasets. For the calibration dataset X, we follow [14] to use 128 random 2048 token segments from the C4 dataset.

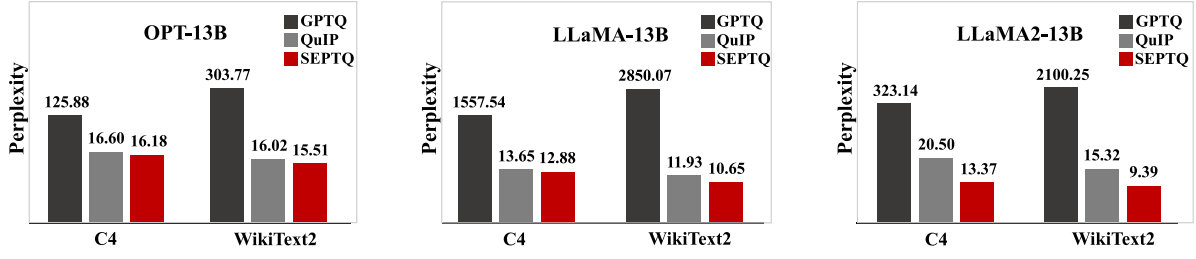


Figure 5: The perplexity results of the OPT-13B, LLaMA-13B and LLaMA2-13B models on C4 and WikiText2 datasets (The lower value is better). For GPTQ and QuIP, they are 2-bit models. And for SEPTQ, it is a 2.01-bit model.

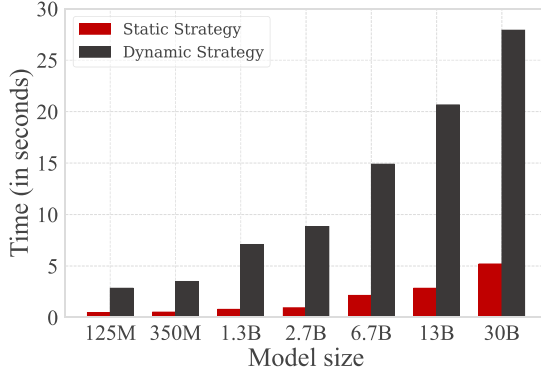


Figure 6: The efficiency results by using the static strategy and dynamic strategy respectively.

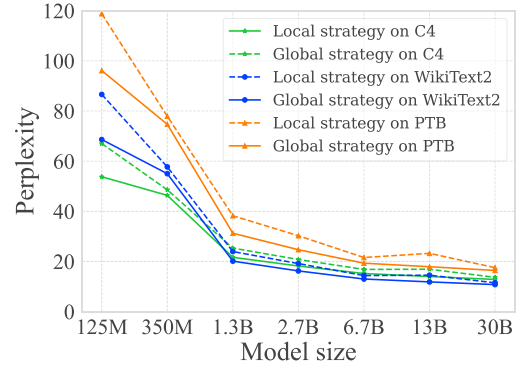


Figure 7: The effectiveness results by using the global strategy and local strategy respectively.

5.1.2 Baselines. We compare our method with the strong post-training quantization methods. (1) **GPTQ** [14] uses a layer-wise quantization technology based on approximate second-order information. (2) **QuIP** [5] adopts a post-training quantization method, which introduces a quantization method of incoherent processing. (3) We also compare with **OmniQuant** [35], **AWQ** [24], **AQLM** [10], **SpQR** [21], due to space and time limit, we show the results in Appendix C with only the perplexity results of the OPT model family on the WikiText2 dataset.

5.1.3 Evaluation Metrics. For perplexity experiments, we use *perplexity* as the metric to evaluate the performance of different models. Lower perplexity indicates that the quantization model is better. For zero-shot experiments, we use *accuracy* to assess the transfer ability of different models. Higher accuracy indicates that the quantization model performs more excellent.

5.2 Implementation Details

We conduct experiments at three different compression levels: 2-bit, 3-bit and 4-bit. As our method needs to reserve a very small part of important weight parameters ($p\%$ of the weight matrix, and $p = 1$), strictly speaking our method is 2.1-bit, 3.1-bit and 4.1-bit. We select the quantization parameters S and Z using a grid searching strategy between the minimum and the maximum vales of each weight matrix with the goal to minimize $\|\mathbf{W} - \widehat{\mathbf{W}}\|_F^2$. We set the blocksize

$B = 128$ consistently. All the experiments are implemented on NVIDIA A800 GPUs and completed within 3 hours.

5.3 Result Analysis of Perplexity Experiments

Tables 1, 2, 3 show the performance of the OPT model family on the C4, WikiText2 and PTB datasets, and the best results are highlighted in bold. According to these tables, it can be seen that in various model sizes, our proposed SEPTQ method obviously outperforms other strong baselines in most cases, especially in extremely low-bit quantization (2-bit). Specifically, when compressing the OPT-66B model to 2 bits on the C4, WikiText2 and PTB datasets, our proposed SEPTQ can achieve 11.96, 9.93 and 14.93 in perplexity respectively, while the perplexity scores of GPTQ and QuIP are only 253.49, 619.32, 630.42 and 17.64, 13.86, 17.50 respectively. It is worth mentioning that the performance of our model at 2-bit level is close to the full precision method. In summary, all these results demonstrate that SEPTQ can compress models across various parameter ranges to very low quantization levels while maintaining robust model performance.

Strictly speaking, in the above comparison SEPTQ requires to reserve an extra 1% important weight parameters, which seems a little unfair and unreasonable. To further verify the effectiveness of SEPTQ, we set the ratio of important weight parameters to 0.1%, i.e., $p = 0.1\%$, then our model at 2-bit level becomes a 2.01-bit model. Figure 5 shows the perplexity results of the OPT-13B, LLaMA-13B and LLaMA2-13B models on C4 and WikiText2 datasets. For GPTQ

Table 4: The zero-shot performance of the LLaMA model family on the different datasets (The higher value is better). “Full” means the full precision method (the original model). For SEPTQ, it is exactly 2.1-bit and 3.1-bit respectively.

Method	Bits	Size	Accuracy (%)					
			PIQA	ARC-e	ARC-c	HellaSwag	Winogrande	Avg.
Full	16	7B	77.31	52.48	41.38	72.96	67.09	62.24
GPTQ	2		47.88	25.55	27.90	26.09	48.54	35.19
QuIP			62.13	36.20	27.56	44.76	51.85	44.50
SEPTQ			65.83	40.70	28.49	51.16	58.40	48.91
GPTQ	3		73.94	45.03	36.43	65.77	60.38	56.31
QuIP			75.57	49.71	37.63	68.24	62.51	58.73
SEPTQ			76.22	52.23	39.33	69.20	65.50	60.50
Full	16	13B	79.11	59.90	44.53	76.23	70.01	65.96
GPTQ	2		48.97	25.00	27.56	24.90	52.09	35.70
QuIP			69.42	44.23	31.74	55.47	56.99	51.57
SEPTQ			74.97	50.80	37.12	66.76	68.43	59.62
GPTQ	3		75.52	52.02	39.16	70.38	65.19	60.45
QuIP			77.20	57.79	41.55	73.23	67.96	63.55
SEPTQ			78.62	57.03	42.06	74.61	68.75	64.21
Full	16	30B	80.09	58.92	45.48	79.21	72.93	67.33
GPTQ	2		50.27	26.52	27.73	25.53	45.46	35.10
QuIP			71.55	46.09	34.22	61.90	60.14	54.78
SEPTQ			76.99	54.00	41.38	72.09	70.88	63.07
GPTQ	3		77.64	52.15	40.44	74.37	71.03	63.13
QuIP			77.97	54.59	42.91	75.54	70.88	64.38
SEPTQ			80.30	57.53	44.80	78.21	73.01	66.77

and QuIP, they are 2-bit models. And for SEPTQ, it is a 2.01-bit model. From this figure, we can observe that the proposed SEPTQ performs better than GPTQ and QuIP consistently. Specifically, for the results of the LLaMA2-13B model on C4 and WikiText2 datasets, the perplexity scores of SEPTQ are better than those of GPTQ 309.63 and 2090.61 respectively, and better than those of QuIP 7.13 and 5.93 respectively. These results further validate that SEPTQ has the ability to compress the model with satisfactory performance only by reserving a small amount of additional important weight parameters.

5.4 Result Analysis of Zero-Shot Experiments

Table 4 provides the accuracy results of the LLaMA model family from 7 billions to 30 billions on various datasets including PIQA, ARC-easy, ARC-challenge, HellaSwag, and Winogrande. The best results are highlighted in bold. The experimental outcomes unequivocally demonstrate that our SEPTQ method consistently outperforms other strong baselines across various datasets. Specifically, for the LLaMA-7B, LLaMA-13B, and LLaMA-30B models in the 2-bit quantization level, the SEPTQ method increases the average accuracy by 4.41%, 8.05%, and 8.29% respectively compared with the QuIP method, and increases the average accuracy by 13.72%, 23.92% and 27.97% respectively compared with the GPTQ method. For the LLaMA-13B model on the Winogrande dataset, the accuracy

of SEPTQ at the 2-bit level (68.43%) is even better than those of GPTQ (65.19%) and QuIP (67.96%) at the 3-bit level, which further demonstrates the superiority of our proposed SEPTQ method. In addition, it also can be seen that by reserving additional 0.1 bit to record the important weight parameters, the performance gains of SEPTQ on these zero-shot tasks are significant, particularly for 2-bit quantization and larger models. All in all, our proposed SEPTQ has the ability to obtain satisfactory accuracy results while compressing models to very low quantization levels across a broad spectrum of parameters in zero-shot tasks.

5.5 Comparison of Different Quantization Location Determining Strategies

(1) For the efficiency, Figure 6 shows the running time of calculating the important scores for each first layer (including six weight matrices) of the OPT model family by using the static strategy and dynamic strategy respectively. From the results, it can be seen that our proposed static strategy runs much faster than the dynamic strategy across different model sizes. This is because that the dynamic strategy requires to updating weight parameters column-by-column, thus causing additional time cost. (2) For the effectiveness, Figure 7 shows the perplexity results of the OPT model family on C4, WikiText2 and PTB datasets by using the global strategy and local strategy respectively. From the results, it can be observed that

Table 5: Running time (in seconds) of different post-training quantization methods on OPT models with WikiText2 at 2 bits.

Method	125M	350M	1.3B	2.7B	6.7B	13B	30B	66B
GPTQ	34	91	216	384	727	1311	2851	5926
QuIP	54	127	201	475	993	2429	7788	16915
SpQR	92	195	376	735	1440	2590	5111	10354
SEPTQ	43	116	263	458	883	1606	3126	6598

our proposed global strategy consistently performs better than the local strategy across different model sizes. The reason is that the distribution of importance weights is irregular and concentrated in certain areas, thus making it difficult for the local strategy to capture appropriate quantitative locations. (3) Based on the above experiments, we can get that using the static global strategy to determine the quantization location is a good and reasonable choice.

5.6 Comparison of Running Time among Different PTQ Methods

The speed of the quantization algorithm is extremely important for post-training quantization. We compared the running time of multiple algorithms, including GPTQ[14], QuIP[5], and SpQR[21]. We conduct experiments with OPT on WikiText2 at 2 bits and list the runtime (in seconds) of different PTQ methods as Table 5. The results show that our method is a little slower than GPTQ, due to the additional determining the quantization location procedure. But our method is faster than other baselines, especially in larger-scale models like OPT 30B and 66B. In summary, our method can obtain satisfactory performance with acceptable time consumption.

6 Conclusion

In this paper, we propose a simple yet effective paradigm for post-training quantization paradigm for large language models, named SEPTQ. The proposed SEPTQ consists of two components: determining the quantization location and quantizing the model weights. For determining the quantization location, we employ a static global strategy which guarantees the effectiveness and efficiency of the method simultaneously. For quantizing the model weights, we leverage the mask matrix that represents the important locations to constrain the derivation of update formula for unquantized weights, thereby precisely compensate the rounding error caused by quantized weights. Extensive experiments on large language models across different bit levels demonstrate the superiority of our method over other strong baselines. In future work, we plan to explore more accurate quantization location strategies and efficient parameter adaptation methods to improve the current model.

7 Acknowledgements

This work was supported by Liaoning Binhai Laboratory Project (No. LBLF-2023-01), National Natural Science Foundation of China (No. 62106035, 62206038), and Chunhui Project Foundation of the Education Department of China (No. HZKY20220419). We also would like to thank Dalian Ascend AI Computing Center and Dalian Ascend AI Ecosystem Innovation Center for providing inclusive computing power and technical support.

References

- [1] Ron Banner, Yuri Nahshan, and Daniel Soudry. 2019. Post training 4-bit quantization of convolutional networks for rapid-deployment. In *Conference on Neural Information Processing Systems (NeurIPS)*. 7948–7956.
- [2] Sumithra Bhakthavatsalam, Daniel Khashabi, Tushar Khot, Bhavana Dalvi Mishra, Kyle Richardson, Ashish Sabharwal, Carissa Schoenick, Oyvind Tafjord, and Peter Clark. 2021. Think you have Solved Direct-Answer Question Answering? Try ARC-DA, the Direct-Answer AI2 Reasoning Challenge. *CoRR* abs/2102.03315 (2021).
- [3] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. 2020. PIQA: Reasoning about Physical Commonsense in Natural Language. In *AAAI Conference on Artificial Intelligence (AAAI)*. 7432–7439.
- [4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. In *Conference on Neural Information Processing Systems (NeurIPS)*. 1877–1901.
- [5] Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher De Sa. 2023. QuIP: 2-Bit Quantization of Large Language Models With Guarantees. In *Conference on Neural Information Processing Systems (NeurIPS)*.
- [6] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. *CoRR* abs/2208.07339 (2022).
- [7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *North American Chapter of the Association for Computational Linguistics (NAACL)*. 4171–4186.
- [8] Shaojin Ding, Phoenix Meadowlark, Yanzhang He, Lukasz Lew, Shivani Agrawal, and Oleg Rybakov. 2022. 4-bit Conformer with Native Quantization Aware Training for Speech Recognition. In *Conference of the International Speech Communication Association (INTERSPEECH)*. 1711–1715.
- [9] Xin Ding, Xiaoyu Liu, Yun Zhang, Zhiyun Tu, Wei Li, Jie Hu, Hanting Chen, Yehui Tang, Zhiwei Xiong, Baoqun Yin, and Yunhe Wang. 2023. CBQ: Cross-Block Quantization for Large Language Models. *CoRR* abs/2312.07950 (2023).
- [10] Vage Egiazarian, Andrei Panferov, Denis Kuznetsov, Elias Frantar, Artem Babenko, and Dan Alistarh. 2024. Extreme Compression of Large Language Models via Additive Quantization. In *International Conference on Machine Learning (ICML)*.
- [11] Jun Fang, Ali Shafiee, Hamzah Abdel-Aziz, David Thorsley, Georgios Georgiadis, and Joseph Hassoun. 2020. Post-training Piecewise Linear Quantization for Deep Neural Networks. In *European Conference on Computer Vision (ECCV)*. 69–86.
- [12] Elias Frantar and Dan Alistarh. 2022. Optimal Brain Compression: A Framework for Accurate Post-Training Quantization and Pruning. In *Conference on Neural Information Processing Systems (NeurIPS)*.
- [13] Elias Frantar and Dan Alistarh. 2024. QMoE: Sub-1-Bit Compression of Trillion Parameter Models. In *Conference on Machine Learning and Systems (MLSys)*.
- [14] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2022. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. *CoRR* abs/2210.17323 (2022).
- [15] Babak Hassibi and David G. Stork. 1992. Second Order Derivatives for Network Pruning: Optimal Brain Surgeon. In *Conference on Neural Information Processing Systems (NeurIPS)*. 164–171.
- [16] Wei Huang, Yangdong Liu, Haotong Qin, Ying Li, Shiming Zhang, Xianglong Liu, Michele Magno, and Xiaojuan Qi. 2024. BiLLM: Pushing the Limit of Post-Training Quantization for LLMs. In *International Conference on Machine Learning (ICML)*.
- [17] Itay Hubara, Yuri Nahshan, Yair Hanani, Ron Banner, and Daniel Soudry. 2021. Accurate Post Training Quantization With Small Calibration Sets. In *International Conference on Machine Learning (ICML)*. 4466–4475.
- [18] Minsoo Kim, Sihwa Lee, Sukjin Hong, Du-Seong Chang, and Jungwook Choi. 2022. Understanding and Improving Knowledge Distillation for Quantization Aware Training of Large Transformer Encoders. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 6713–6725.
- [19] Eldar Kurtic, Daniel Campos, Tuan Nguyen, Elias Frantar, Mark Kurtz, Benjamin Fineran, Michael Goin, and Dan Alistarh. 2022. The Optimal BERT Surgeon:

- Scalable and Accurate Second-Order Pruning for Large Language Models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 4163–4181.
- [20] Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, et al. 2022. BLOOM: A 176B-Parameter Open-Access Multilingual Language Model. *CoRR abs/2211.05100* (2022).
- [21] Dohyeok Lee, Seungyub Han, Taehyun Cho, and Jungwoo Lee. 2023. SPQR: Controlling Q-ensemble Independence with Spiked Random Model for Reinforcement Learning. In *Conference on Neural Information Processing Systems (NeurIPS)*.
- [22] Jung Hyun Lee, Jeonghoon Kim, Se Jung Kwon, and Dongsoo Lee. 2023. FlexRound: Learnable Rounding based on Element-wise Division for Post-Training Quantization. In *International Conference on Machine Learning (ICML)*. 18913–18939.
- [23] Yuhang Li, Ruihao Gong, Xu Tan, Yang Yang, Peng Hu, Qi Zhang, Fengwei Yu, Wei Wang, and Shi Gu. 2021. BRECO: Pushing the Limit of Post-Training Quantization by Block Reconstruction. In *International Conference on Learning Representations (ICLR)*.
- [24] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. In *Conference on Machine Learning and Systems (MLSys)*.
- [25] Shih-Yang Liu, Zechun Liu, and Kwang-Ting Cheng. 2023. Oscillation-free Quantization for Low-bit Vision Transformers. In *International Conference on Machine Learning (ICML)*. 21813–21824.
- [26] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *CoRR abs/1907.11692* (2019).
- [27] Zechun Liu, Barlas Oguz, Changsheng Zhao, Ernie Chang, Pierre Stock, Yashar Mehdad, Yangyang Shi, Raghuraman Krishnamoorthi, and Vikas Chandra. 2024. LLM-QAT: Data-Free Quantization Aware Training for Large Language Models. In *Annual Meeting of the Association for Computational Linguistics (ACL)*. 467–484.
- [28] Zhenhua Liu, Yunhe Wang, Kai Han, Wei Zhang, Siwei Ma, and Wen Gao. 2021. Post-Training Quantization for Vision Transformer. In *Conference on Neural Information Processing Systems (NeurIPS)*. 28092–28103.
- [29] Mitchell P. Marcus, Grace Kim, Mary Ann Marcinkiewicz, Robert MacIntyre, Ann Bies, Mark Ferguson, Karen Katz, and Britta Schasberger. 1994. The Penn Treebank: Annotating Predicate Argument Structure. In *Human Language Technology*.
- [30] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer Sentinel Mixture Models. In *International Conference on Learning Representations (ICLR)*.
- [31] Markus Nagel, Rana Ali Amjad, Mart van Baalen, Christos Louizos, and Tijmen Blankevoort. 2020. Up or Down? Adaptive Rounding for Post-Training Quantization. In *International Conference on Machine Learning (ICML)*. 7197–7206.
- [32] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1 (2019).
- [33] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research (JMLR)* 21 (2020), 140:1–140:67.
- [34] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2021. WinoGrande: an adversarial winograd schema challenge at scale. *Commun. ACM* 64, 9 (2021), 99–106.
- [35] Wenqi Shao, Mengzhao Chen, Zhaoyang Zhang, Peng Xu, Lirui Zhao, Zhiqian Li, Kaipeng Zhang, Peng Gao, Yu Qiao, and Ping Luo. 2024. OmniQuant: Omnidirectionally Calibrated Quantization for Large Language Models. In *International Conference on Learning Representations (ICLR)*.
- [36] Shyam Anil Tailor, Javier Fernández-Marqués, and Nicholas Donald Lane. 2021. Degree-Quant: Quantization-Aware Training for Graph Neural Networks. In *International Conference on Learning Representations (ICLR)*.
- [37] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. *CoRR abs/2302.13971* (2023).
- [38] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Conference on Neural Information Processing Systems (NeurIPS)*. 5998–6008.
- [39] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. 2023. BitNet: Scaling 1-bit Transformers for Large Language Models. *CoRR abs/2310.11453* (2023).
- [40] Guangxuan Xiao, Ji Lin, Mickaël Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. In *International Conference on Machine Learning (ICML)*. 38087–38099.
- [41] Yuzhuang Xu, Xu Han, Zonghan Yang, Shuo Wang, Qingfu Zhu, Zhiyuan Liu, Weidong Liu, and Wanxiang Che. 2024. OneBit: Towards Extremely Low-bit Large Language Models. *CoRR abs/2402.11295* (2024).
- [42] Zhewei Yao, Reza Yazdani Aminabadi, Minjia Zhang, Xiaoxia Wu, Conglong Li, and Yuxiong He. 2022. ZeroQuant: Efficient and Affordable Post-Training Quantization for Large-Scale Transformers. In *Conference on Neural Information Processing Systems (NeurIPS)*.
- [43] Zhihang Yuan, Lin Niu, Jiawei Liu, Wenyu Liu, Xinggang Wang, Yuzhang Shang, Guangyu Sun, Qiang Wu, Jiaxiang Wu, and Bingzhe Wu. 2023. RPTQ: Reorder-based Post-training Quantization for Large Language Models. *CoRR abs/2304.01089* (2023).
- [44] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. HellaSwag: Can a Machine Really Finish Your Sentence?. In *Annual Meeting of the Association for Computational Linguistics (ACL)*. 4791–4800.
- [45] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona T. Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. 2022. OPT: Open Pre-trained Transformer Language Models. *CoRR abs/2205.01068* (2022).
- [46] Xunyu Zhu, Jian Li, Yong Liu, Can Ma, and Weiping Wang. 2023. A Survey on Model Compression for Large Language Models. *CoRR abs/2308.07633* (2023).

A The Detailed Proof Procedure of Eq. (4)

Here we give the proof procedure from Eq. (3) to Eq. (4). Given a weight matrix $\mathbf{W}$, the procedure involves quantizing the element at the i -th row and j -th column, and updating the unquantized weights in the i -th row to compensate for the loss. As each row of $\mathbf{W}$ is independent, so altering the values in the i -th row does not affect the outputs of the other rows.

Let the updated i -th row $\mathbf{W}_{i,:}$ be $\widehat{\mathbf{W}}_{i,:}$, then Eq. (3) can be written as:

$$\begin{aligned} s_{i,j} &= \min_{\widehat{\mathbf{W}}_{i,j}} \|\mathbf{W}_{i,:}\mathbf{X} - \widehat{\mathbf{W}}_{i,:}\mathbf{X}\|_F^2 \\ \text{s.t. } \widehat{\mathbf{W}}(i,j) &= \text{quant}(\mathbf{W}(i,j)), \end{aligned} \quad (9)$$

where $\|\cdot\|_F$ is the Frobenius norm, $\widehat{\mathbf{W}}(i,j)$ denotes the matrix $\mathbf{W}$ with only $\mathbf{W}_{i,j}$ is quantized and unquantized elements in the i -th row are updated.

For ease of representation, we use $f(\widehat{\mathbf{W}}_{i,:}) = \|\mathbf{W}_{i,:}\mathbf{X} - \widehat{\mathbf{W}}_{i,:}\mathbf{X}\|_F^2$ to denote the function of $\widehat{\mathbf{W}}_{i,:}$. By leveraging Taylor's expansion on $f(\widehat{\mathbf{W}}_{i,:})$ at $\widehat{\mathbf{W}}_{i,:} = \mathbf{W}_{i,:}$, we can have:

$$\begin{aligned} f(\widehat{\mathbf{W}}_{i,:}) &= f(\mathbf{W}_{i,:}) + (\widehat{\mathbf{W}}_{i,:} - \mathbf{W}_{i,:})^T \Delta E(\mathbf{W}_{i,:}) \\ &\quad + \frac{1}{2}(\widehat{\mathbf{W}}_{i,:} - \mathbf{W}_{i,:})^T \mathbf{H}(\widehat{\mathbf{W}}_{i,:} - \mathbf{W}_{i,:}) + o, \end{aligned} \quad (10)$$

where $\Delta E(\mathbf{W}_{i,:})$ is the first derivative of f with respect to $\widehat{\mathbf{W}}_{i,:}$ at $\widehat{\mathbf{W}}_{i,:} = \mathbf{W}_{i,:}$. The matrix $\mathbf{H}$ is the Hessian matrix which represents the second derivative of f with respect to $\widehat{\mathbf{W}}_{i,:}$ at $\widehat{\mathbf{W}}_{i,:} = \mathbf{W}_{i,:}$, and o is an infinitesimal of higher order.

As $\Delta E(\mathbf{W}_{i,:}) = \mathbf{0}$, $f(\mathbf{W}_{i,:}) = 0$ and $\mathbf{H} = 2\mathbf{X}\mathbf{X}^T$, then Eq. (10) can be written as:

$$f(\widehat{\mathbf{W}}_{i,:}) = \frac{1}{2}(\widehat{\mathbf{W}}_{i,:} - \mathbf{W}_{i,:})\mathbf{H}(\widehat{\mathbf{W}}_{i,:} - \mathbf{W}_{i,:})^T. \quad (11)$$

As $f(\widehat{\mathbf{W}}_{i,:}) = \|\mathbf{W}_{i,:}\mathbf{X} - \widehat{\mathbf{W}}_{i,:}\mathbf{X}\|_F^2$, then substituting Eq. (11) to Eq. (9), we can have:

$$\begin{aligned} s_{i,j} &= \min_{\widehat{\mathbf{W}}_{i,j}} \frac{1}{2}(\widehat{\mathbf{W}}_{i,:} - \mathbf{W}_{i,:})\mathbf{H}(\widehat{\mathbf{W}}_{i,:} - \mathbf{W}_{i,:})^T, \\ \text{s.t. } \widehat{\mathbf{W}}(i,j) &= \text{quant}(\mathbf{W}(i,j)), \end{aligned} \quad (12)$$

By using the Lagrange multiplier method, we can have:

$$\begin{aligned} s_{i,j} &= \min_{\widehat{\mathbf{W}}_{i,j}} \left[\frac{1}{2}(\widehat{\mathbf{W}}_{i,:} - \mathbf{W}_{i,:})\mathbf{H}(\widehat{\mathbf{W}}_{i,:} - \mathbf{W}_{i,:})^T \right. \\ &\quad \left. + \lambda (\widehat{\mathbf{W}}(i,j) - \text{quant}(\mathbf{W}(i,j))) \right]. \end{aligned} \quad (13)$$

By solving the derivatives of $\widehat{\mathbf{W}}_{i,:}$ and λ , and make them equal 0, we can get:

$$s_{i,j} = \frac{(\mathbf{W}_{i,j} - \text{quant}(\mathbf{W}_{i,j}))^2}{2[\mathbf{X}\mathbf{X}^T]_{j,j}^{-1}}. \quad (14)$$

B The Detailed Proof Procedure of Eq. (8)

Here we give the proof procedure from Eq. (1) (with the constraint Eq. (5)) to Eq. (8). The optimization problem posed by the objective function $\arg\min_{\widehat{\mathbf{W}}} \|\mathbf{W}\mathbf{X} - \widehat{\mathbf{W}}\mathbf{X}\|_F^2$ is subject to the constraint $\widehat{\mathbf{W}} = \text{quant}(\mathbf{W}) + \mathbf{M} \odot (\mathbf{W} - \text{quant}(\mathbf{W}))$. For a model with a particularly large weight $\mathbf{W}$ dimension, the time complexity of globally solving

the optimal $\widehat{\mathbf{W}}$ is very high. Therefore, we attempt to decompose the problem. First, we rewrite $\|\mathbf{W}\mathbf{X} - \widehat{\mathbf{W}}\mathbf{X}\|_F^2$ as:

$$\|\mathbf{W}\mathbf{X} - \widehat{\mathbf{W}}\mathbf{X}\|_F^2 = \sum_{i=1}^{d_{\text{row}}} \|\mathbf{W}_{i,:}\mathbf{X} - \widehat{\mathbf{W}}_{i,:}\mathbf{X}\|_F^2, \quad (15)$$

which allows us to address each row's weight optimization independently. Our focus then shifts to minimize the output difference for each row's weight. Specifically, for $\mathbf{W}_{i,:}$, our goal becomes:

$$\arg\min_{\widehat{\mathbf{W}}_{i,:}} \|\mathbf{W}_{i,:}\mathbf{X} - \widehat{\mathbf{W}}_{i,:}\mathbf{X}\|_F^2. \quad (16)$$

According to the proof procedure in Line 1052 - Line 1078, if we introduce the increment $\boldsymbol{\delta} = (\widehat{\mathbf{W}}_{i,:} - \mathbf{W}_{i,:})^T$, where T denotes the transposition, then Eq. (16) can be written as:

$$\arg\min_{\widehat{\mathbf{W}}_{i,:}} \frac{1}{2} \boldsymbol{\delta}^T \mathbf{H} \boldsymbol{\delta}, \quad (17)$$

where $\mathbf{H} = \mathbf{X}\mathbf{X}^T$ denotes the Hessian matrix. Assuming that we are dealing with the j -th value of the i -th row, based on Eq. (5), we know that $\widehat{\mathbf{W}}_{i,j} = \text{quant}(\mathbf{W}_{i,j}) + \mathbf{M}_{i,j}(\mathbf{W}_{i,j} - \text{quant}(\mathbf{W}_{i,j}))$, where $\mathbf{M}$ is mask matrix of $\mathbf{W}$. Then we can have $\boldsymbol{\delta}_j = \widehat{\mathbf{W}}_{i,j} - \mathbf{W}_{i,j} = \mathbf{e}_j^T \boldsymbol{\delta}$, where $\mathbf{e}_j$ represents the unit vector with 1 in the j -th position. This is to say, we can transfer the constraint of Eq. (5) to the constraint:

$$\mathbf{e}_j^T \boldsymbol{\delta} + (\mathbf{W}_{i,j} - \widehat{\mathbf{W}}_{i,j}) = 0. \quad (18)$$

According to Eq. (17) and the constraint $\mathbf{e}_j^T \boldsymbol{\delta} + (\mathbf{W}_{i,j} - \widehat{\mathbf{W}}_{i,j}) = 0$, by using the Lagrange multiplier method, we can have:

$$L(\boldsymbol{\delta}, \lambda) = \frac{1}{2}(\boldsymbol{\delta}^T \mathbf{H} \boldsymbol{\delta}) + \lambda(\mathbf{e}_j^T \boldsymbol{\delta} + (\mathbf{W}_{i,j} - \widehat{\mathbf{W}}_{i,j})). \quad (19)$$

By solving the derivatives of $\boldsymbol{\delta}$ and λ , we can get:

$$\boldsymbol{\delta} = -\frac{\mathbf{W}_{i,j} - \widehat{\mathbf{W}}_{i,j}}{[\mathbf{H}^{-1}]_{jj}} [\mathbf{H}^{-1}]_{:,j}. \quad (20)$$

C Additional Results

We compare our proposed SEPTQ method with several other quantization methods, specifically OmniQuant [35], AWQ [24], AQLM [10] and SpQR [21]. To ensure clarity and due to space and time limitations, we present the detailed perplexity results of the OPT model family on the WikiText2 dataset. Table 7 summarizes these results, with the best-performing figures highlighted in bold for easy reference.

We also have attempted to explore whether it is a trick by observing the output projection matrices of some consecutive linear layers (from the 4th to the 7th layers) as Figure 8. We find that no matter which layer is selected, the similar trend can be observed.

Table 6: Quantization bits of SpQR and AQLM.

MethodBit	125M	1.3B	2.7B	6.7B	13B	30B	66B
SpQR(2)	2.6	2.3	2.3	2.1	2.2	2.1	2.1
SpQR(3)	3.7	3.5	3.5	3.5	3.3	3.5	3.9
SpQR(4)	4.3	4.3	4.1	4.2	4.2	4.3	4.2
AQLM(2)	2.0	2.0	1.9	2.0	2.2	1.9	1.9

Table 7: The perplexity results of the OPT model family on the WikiText2 dataset (The lower value is better). “Full” means the full precision method (the original model). For SEPTQ, it is exactly 2.1-bit, 3.1-bit and 4.1-bit respectively. Since the number of quantization bits of SpQR and AQLM is flexible, we provide the specific number of quantization bits of SpQR and AQLM for each model in Table 6.

Method	Bit	125M	1.3B	2.7B	6.7B	13B	30B	66B
Full	16	27.65	14.63	12.47	10.86	10.13	9.56	9.34
OmniQuant	2	75.43	23.95	18.13	14.43	12.94	11.39	30.84
AWQ		251.84	47.97	28.50	16.20	14.32	12.31	14.54
SpQR		116.94	22.62	16.57	14.31	11.95	10.56	9.99
AQLM		64.88	24.62	17.49	12.91	11.59	10.78	10.37
SEPTQ		68.62	20.11	16.20	12.97	11.82	10.75	9.93
OmniQuant	3	35.66	16.68	13.80	11.65	10.87	10.00	9.83
AWQ		36.74	16.32	13.58	11.41	10.68	9.85	9.60
SpQR		34.37	15.73	12.75	11.17	10.36	9.65	9.32
SEPTQ		33.77	15.46	12.74	11.14	10.33	9.61	9.27
OmniQuant	4	29.45	15.04	12.76	11.03	10.30	9.65	9.65
AWQ		32.28	15.49	12.93	11.30	10.39	9.77	9.61
SpQR		29.37	15.03	12.57	10.91	10.22	9.50	9.37
SEPTQ		29.00	14.88	12.46	10.98	10.22	9.54	9.27

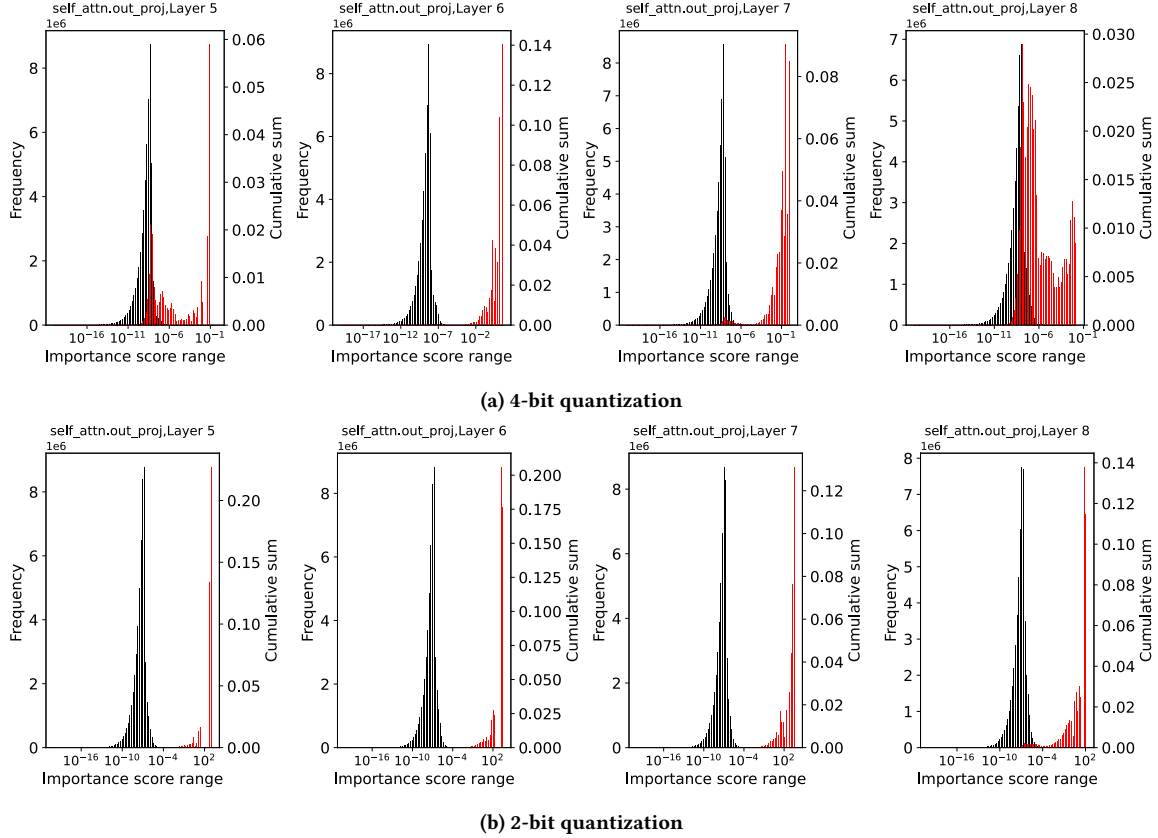


Figure 8: Output projection matrices of consecutive linear layers (4th to 7th Layers) showing consistent trends across layers